

Green Browsing – Comparing Energy Consumption of Modern Web Browsers

Dennis Junger^[0000–0002–9271–2261],
Marie Charlotte Epp^[0009–0001–4777–5378],
Kevin Jacob Bärwald^[0009–0005–5388–4625],
Lea Catharina Nicolaus^[0009–0007–3390–3272],
Ramona Pangerl^[0009–0008–3740–5438],
Stefan Schlegel^[0009–0005–0391–7230], and
Volker Wohlgemuth^[0009–0000–3150–1746]

To Measure is to Know – Lord Kelvin.

Abstract The energy consumption of software applications has become increasingly relevant in sustainable computing, particularly for widely used applications such as web browsers. This study compares the energy consumption and resource utilization of five desktop web browsers using a reproducible measurement framework. A standardized usage scenario was executed while electrical power consumption, CPU utilization, memory usage, disk activity, and network traffic were recorded. The measurements revealed measurable differences between the evaluated browsers. Within the standardized scenario, Google Chrome exhibited the lowest mean electrical power consumption (20.4 W), whereas Microsoft Edge showed the highest (25.2 W), corresponding to an additional 4.8 W or 23.5% compared with Chrome. CPU utilization varied only slightly, while larger differences were observed in memory consumption, disk activity, network communication, and idle behavior. In particular, Opera generated substantially higher disk activity, and Brave exhibited the highest idle power consumption and background network activity. Extrapolated to 300 million users and assuming two hours of daily browser use, the 4.8 W difference between Edge and Chrome would correspond to approximately 1.05 TWh of additional annual electricity demand. These results demonstrate that browser energy efficiency cannot be assessed using a single resource metric and highlight the value of combining electrical power measurements with complementary system-level metrics.

Keywords Green IT · Web Browsers · Sustainable Computing · Benchmarking · Green Coding · Green Software · Blue Angel for software

1 Introduction

The environmental impact of software has gained increasing attention as digitalization continues to expand. In 2020, the information and communication technology (ICT) sector accounted for approximately 4% of global electricity consumption, with end-user devices responsible for the largest share [20]. While research on software energy efficiency has largely focused on servers and data centers, comparatively little attention has been paid to desktop applications despite their widespread use and cumulative energy demand [2, 33]. Within the field of *Green Software Engineering*, reducing software-induced energy consumption requires reliable and reproducible measurement approaches [23, 14]. Web browsers are a particularly relevant target for such analyses because they are among the most frequently used desktop applications and exhibit substantial differences in architecture, rendering engines, and resource-management strategies [36, 5, 32]. Scenario-based measurement of software-induced energy consumption has a long tradition within the environmental informatics community. Dick et al. [6] introduced a standardized measurement and rating procedure for desktop PCs and servers at EnviroInfo 2011, defining reusable usage scenarios per software category – web browsers among them – and subsequently applied the approach to web applications [7]. This line of work later informed the criteria of the Blue Angel eco-label for resource- and energy-efficient software products [8]. More recently, Kleinert and Arndt [18] applied comparable scenario-based power measurements to web application load distribution. The present study builds directly on this methodological tradition rather than proposing a new one.

This study investigates the energy efficiency of five widely used desktop browsers using an adapted measurement framework at HTW Berlin [17] inspired by the Blue Angel [8] methodology in the HTW Green Coding Course [16].

Compared with the earlier work, this study makes four key contributions. First, the browser landscape has changed substantially since the earlier measurements. Four of the five browsers evaluated in this study use the Chromium/Blink rendering engine, enabling a direct investigation of whether a common rendering engine is associated with comparable resource-use and power-consumption patterns — a configuration that did not exist when the earlier studies were conducted. Second, external electrical power measurements are synchronized with system-level monitoring of processor, memory, disk, and network utilization. This enables observed power profiles to be related to specific resource classes rather than reported only as aggregate consumption. Third, browser background communication is captured at the level of the Domain Name System (DNS), an aspect that has received limited attention in browser energy studies, while idle operation is measured separately from active use. Fourth, the complete raw measurement data for all experimental runs are published openly (see Section 3), enabling independent reanalysis and supporting potential use of the dataset by browser vendors in the context of certification for the Blue Angel software label.

All configurations were measured in 20 repeated runs per browser within a tightly controlled local test environment. On this basis, the study quantifies differences in browser energy consumption, identifies the resource-related drivers of power demand—central processing unit (CPU), memory, disk, and network utilization—and assesses the

contribution of background activity during idle operation. Accordingly, it addresses the following research questions:

- **RQ1:** How do popular desktop web browsers differ in their electrical power consumption under a standardized usage scenario?
- **RQ2:** How are CPU utilization, memory consumption, disk activity, and network traffic related to the observed power-consumption profiles?
- **RQ3:** How do web browsers behave during idle operation in terms of background activity and power demand?

2 Methodology

2.1 Operating System Selection

The Blue Angel methodology recommends performing measurements in a standardized environment containing only the software required for the experiment to minimize external influences [8]. The experiments were therefore conducted on Tiny11, a streamlined Windows 11 Pro (version 25H2, OS Build 26200.6584) installation that removes unnecessary system components and background services. Microsoft Windows was chosen because it is the dominant desktop operating system and therefore represents a relevant platform for evaluating desktop applications.¹

Before the measurement series of each browser, the Tiny11 installation was restored from a fixed reference image to ensure identical starting conditions across browsers and to minimize background interference. Within a series, the 20 runs of a given browser were executed consecutively on this restored image.

2.2 Selection and Configuration of the Browsers

To ensure practical relevance, the five most widely used desktop browsers according to current download statistics were selected: Google Chrome, Mozilla Firefox, Microsoft Edge, Opera, and Brave [4]. Firefox was included as the only browser based on the Gecko rendering engine², whereas the remaining browsers are Chromium-based and use the Blink engine.³ The latest available 64-bit versions at the time of the measurements were used: Chrome 143.0.7499.170 [9], Firefox 146.0.1 [22], Edge 143.0.3650.96 [21], Opera 126.0.5750.43 [31], and Brave 1.85.120 [3].

To ensure comparable measurement conditions, each browser was installed in a clean environment and reset to its default configuration before testing. Automatic updates were

¹ <https://electroiq.com/stats/windows-statistics/>

² <https://developer.mozilla.org/en-US/docs/Glossary/Gecko>

³ <https://www.chromium.org/blink/>

disabled, and no extensions, add-ons, or plug-ins were installed to prevent additional background activity during the measurements.

2.3 Methodological Standardization of User Scenarios

The experimental procedure was designed so that all browsers were subjected to the same scenarios. Since different browsers support different keyboard shortcuts and user interactions, the user scenarios were adapted depending on the specific browser context, while their functional objectives remained unchanged.

2.4 Measurement Setup

The measurement setup follows the architecture proposed by [17], which itself is based on the energy measurement methodology defined by the Blue Angel environmental label for energy-efficient software products [8] and further integrated into the Green Coding course described by [15]. The underlying methodology is rooted in the work of [11], who emphasize the use of standardized and reproducible usage scenarios for the comparative assessment of software resource and energy efficiency. The setup comprises three dedicated systems: a *Web Server System*, a *Measurement Client*, and a *Monitoring System* 1. This separation enables reproducible, network-independent energy measurements under controlled operating conditions.

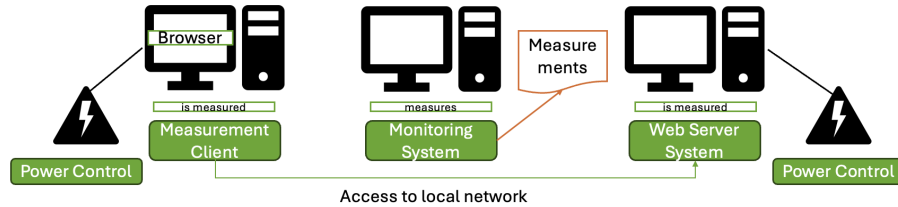


Fig. 1: Measurement setup

The Web Server System hosts a locally deployed test website developed specifically for this study. The website implements a set of interaction scenarios that emulate common browser activities and generate controlled computational workloads. These scenarios form the basis of the standardized usage profile described in Section 2.5.

The Measurement Client executes the browser under test and performs automated interactions with the test website using Actiona [1], an open-source cross-platform automation tool that replays predefined sequences of keyboard and mouse events. It was chosen because it operates at the level of the operating system's input layer rather than through a browser-specific automation interface such as WebDriver [34], so that the

identical interaction script can be executed against all five browsers without introducing browser-specific instrumentation overhead into the measurement.

Following the measurement setup proposed by Junger et al. [17], which is based on the Blue Angel measurement methodology [8], a GUDE Expert Power Control 1202 [12] power meter was installed at the system under test (SUT) and sent the data to the Data Aggregator via Simple Network Management Protocol (SNMP) accumulated at 1 Hz with the python script [10]. Following data acquisition, the electrical power measurements, Windows Performance Monitor logs, and Actiona event logs were synchronized offline using their timestamps. At the beginning of every measurement, the timestamps were synchronized via a Time (NTP) Server in the university network. In addition, Windows Performance Monitor records hardware-utilization metrics throughout each measurement run. The client system is equipped with an Intel Core i5-10500 processor (6 cores, 12 threads), 8 GB DDR4-3200 memory, and a 512 GB NVMe SSD, running a modified Windows 11 installation (Tiny11 [24]). Tiny11 is a modified, resource-optimized variant of Windows 11 in which numerous preinstalled applications, optional system components, and background services are removed or disabled. Compared with a standard Windows 11 installation, it omits a wide range of productivity, multimedia, gaming, communication, and cloud integration applications (e.g., Microsoft Office Hub, Xbox services, Mail and Calendar, Maps, Media Player, Edge, and OneDrive) [24]. As a result, storage and memory requirements are reduced, and integrated cloud, telemetry, and AI-related features, such as Copilot, may also be omitted depending on the specific build.

The hardware and software configuration remained unchanged for all experiments. The test server and the SUT are connected via a dedicated network switch. This configuration provides a stable local network path between the two systems, thereby reducing variability arising from dynamic routing or external network conditions. Both systems nevertheless retain access to the Internet.

DNS requests for external resources are resolved through a Pi-hole instance. Pi-hole is an open-source, network-level DNS resolver that can filter requests using blocklists of advertising and tracking domains while logging each received query. In this study, its filtering functionality is not used to modify browser behavior. Instead, Pi-hole functions as a passive DNS monitoring component, recording the external hostnames contacted by each browser and the frequency of corresponding DNS queries in its query log.

Because Pi-hole records DNS queries rather than complete network traffic, the resulting logs can identify potential external communications without capturing the content of those exchanges. Hosting the test website locally further ensures comparable network conditions across browser experiments and reduces the likelihood that uncontrolled external traffic affects the measured energy consumption.

The Monitoring System records the energy consumption of both the Measurement Client and the Web Server System. Electrical power is measured using two dedicated power-metering power strips, allowing independent acquisition of the energy demand of the system under test and the server infrastructure. This separation prevents measurement interference and improves result accuracy. Furthermore, DNS traffic generated by the Measurement Client is captured using Pi-hole, enabling detailed analysis of browser-specific request behavior at the DNS level.

2.5 Usage Scenario

Standardized interaction scenarios are recommended for comparative evaluations of software energy efficiency, as they ensure reproducibility and comparability across software products [11]. A standardized usage scenario was developed to enable a consistent, reproducible, and practice-oriented assessment of browser energy consumption and resource utilization.

Previous studies have shown that web browser energy consumption is strongly influenced by specific interaction patterns, including page loading, multimedia playback, scrolling, rendering operations, JavaScript execution, and Document Object Model (DOM) manipulations [32, 5]. These activities induce characteristic workload peaks and represent some of the most energy-intensive phases during browser operation. Consequently, the developed scenario incorporates representative browser actions covering these phases, including opening and closing regular and private tabs, video playback, scrolling, canvas rendering, DOM manipulations, and computationally intensive JavaScript tasks. Each scripted action was chosen as a proxy for a specific class of everyday browsing behavior: tab opening/closing and tab churn emulate multitasking across several open websites or services, as is typical when switching between email, search, and streaming or social media tabs; video playback stands in for the sustained decoding and GPU load generated by streaming platforms; scrolling reproduces the continuous layout and paint operations triggered when browsing news feeds, social media timelines, or long articles; canvas rendering approximates graphics-heavy content such as interactive maps, dashboards, or web-based games; and the DOM manipulation and computationally intensive JavaScript tasks represent the scripting overhead of modern single-page applications, dynamic web forms, and client-side data processing common on contemporary websites.

To best reflect everyday browser use, the project team held a focus group to discuss general private browsing behavior and develop a usage scenario that was as comprehensive as possible. The selected workload should therefore be interpreted as a representative benchmark of common browser activities created by a group of 20- to 40-year-old informatics-related users rather than a complete model of everyday web usage of the average human.

To provide controlled and identical workloads, several of these interactions are executed on dedicated pages hosted by the local Web Server System. The scenario is fully automated using Actiona (v3.9.0) [1]. User interactions are executed automatically based on predefined workflows, while short delays between actions are set to react to different responses of the browser and to have a measurable amount of interaction time, such as, for example, video playback time. Figure 2 presents a schematic overview of the interaction sequence. Each browser was evaluated in 20 consecutive measurement runs. Although the Blue Angel methodology specifies a minimum of ten repetitions for software energy measurements [8], the number of measurement runs was doubled to improve statistical robustness and reduce the influence of random fluctuations. As described in Section 2.1, the reference image was restored before each browser series. Before each measurement run, system clocks were synchronized, and a 60 s cooldown period was applied to minimize background activity. Each measurement run lasted

120 s and consisted primarily of keyboard-driven interactions to reduce input variability. During execution, detailed timestamp logs were generated, enabling the correlation of user interactions with hardware utilization and energy measurements. After each measurement run, the browser cache was cleared to prevent cached resources or other client-side state from influencing subsequent repetitions. Since the test website was hosted locally and no caching proxy or server-side cache was deployed within the test network, clearing the browser cache was sufficient to prevent cache-related effects and maintain reproducible loading conditions across runs. In addition to the standardized usage scenario, baseline measurements without browser execution and idle measurements with an open but inactive browser were conducted to isolate the energy overhead attributable to browser operation and user interaction. The complete interaction sequence executed during each measurement run is illustrated in Figure 2.

2.6 Data Processing

The analysis was based on the measurement methodology defined in the Blue Angel criteria for energy- and resource-efficient software products (DE-UZ 215) [8]. For each browser, two types of measurements were performed: a Base measurement, in which only the browser was opened without executing any user interactions, and a Scenario measurement, during which the predefined usage scenario was executed. In addition, a single Idle Base measurement was recorded before all browser-specific measurements. During this measurement, the test system remained in an idle state without any browser or additional application running, providing a baseline for the system's energy consumption. To isolate the energy demand attributable to browser execution, the Idle Base measurement was subtracted from both the Base and Scenario measurements on a per-timestamp basis. This baseline correction removes the system's idle energy consumption from all browser-specific measurements. Due to temporal fluctuations and measurement noise, the corrected values occasionally resulted in negative power values.

The recorded measurement data, including electrical power consumption, CPU utilization, memory usage, and DNS activity, were processed using a custom analysis script. For each metric, the corresponding time series from repeated measurement runs were extracted, aligned to a common time axis through resampling, and aggregated to enable direct comparisons between browsers. Descriptive statistics, including the mean, median, and standard deviation, were computed for each metric. Furthermore, averaged time-series plots were generated and annotated with the timestamps of the user interactions shown in Figure 2. These annotations facilitate the identification of relationships between specific browser actions and corresponding changes in energy consumption or resource utilization. Finally, the processed data were exported as both statistical summaries and visualizations for subsequent evaluation.

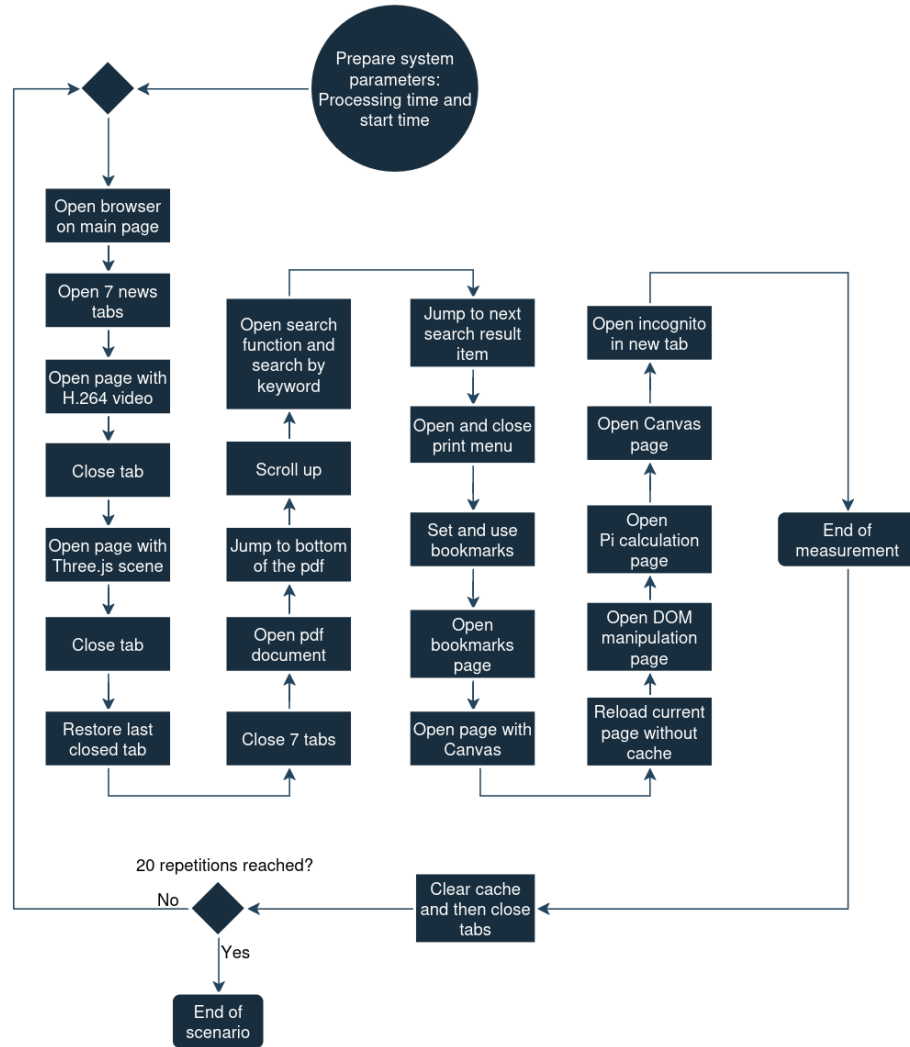


Fig. 2: Schematic representation of the usage scenario.

3 Measurement Results

The following section presents the measurement results. The corresponding raw measurement logs are publicly available for each browser under test:

- Baseline [28] via <https://doi.org/10.5281/zenodo.21081950>
- Brave [25] via <https://doi.org/10.5281/zenodo.21139042>
- Chrome [29] via <https://doi.org/10.5281/zenodo.21082941>
- Edge [27] via <https://doi.org/10.5281/zenodo.21139455>

- Firefox [30] via <https://doi.org/10.5281/zenodo.21138748>
- Opera [26] via <https://doi.org/10.5281/zenodo.21139326>

3.1 Idle Base Measurement

During idle operation, the spider web graph in Figure 4 shows that the browsers differ mainly in network- and disk-related behavior, while the remaining metrics are comparatively close to each other. Brave exhibits the highest network activity, accounting for almost 90% of the total network activity across the five browsers, and Opera shows a pronounced peak in disk activity with roughly 60% of the total. In the other metrics, the differences are smaller: Chrome has the highest CPU utilization, Brave leads in DNS requests, and Firefox is slightly above the others in random access memory (RAM) usage. Overall, the idle measurements indicate that browser resource behavior is not uniform even without active user interaction, with network and disk usage showing the clearest separation between browsers.

The idle power comparison shows that Brave has the highest power consumption, while Firefox and Opera remain slightly below the idle baseline. Chrome is almost identical to the baseline value, whereas Edge shows the second-highest power consumption above idle. These results suggest that even in idle mode, browser-specific background activity can still influence power demand.

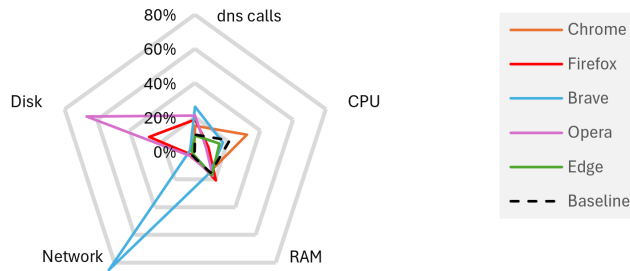


Fig. 3: Idle resource profile across five browsers. Each axis represents one metric, normalized to the maximum observed value across all browsers (i.e., the outermost ring equals 80% of the highest measured value per metric). Power consumption is excluded from this diagram and shown separately.

3.2 Electrical Power

During the standardized usage scenario, Chrome showed the lowest mean power consumption, while Edge showed the highest. The spread across browsers was modest, but

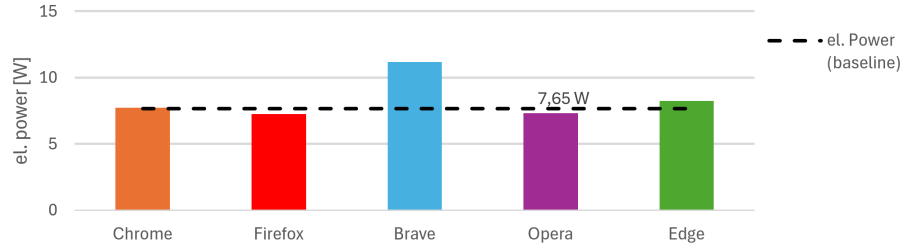


Fig. 4: Idle power consumption comparison across browsers, including the system baseline.

the temporal curves reveal browser-specific peak behavior during video playback, tab management, search operations, and canvas animation. Table 1 and Figure 5 summarize these differences.

Table 1: Mean electrical power consumption of the browsers during the standardized usage scenario (mean $\pm$ SD, $n = 20$). Corrected values are calculated by subtracting the baseline consumption of 7.65 W.

Browser	Mean Power [W]	Corrected Power [W]
Google Chrome	20.4 ± 0.9	12.8
Mozilla Firefox	20.6 ± 0.8	13.0
Opera	21.8 ± 0.8	14.2
Brave	23.7 ± 1.9	16.1
Microsoft Edge	25.2 ± 1.3	17.6

Figure 5 illustrates the mean power consumption across all browsers during the scenario.

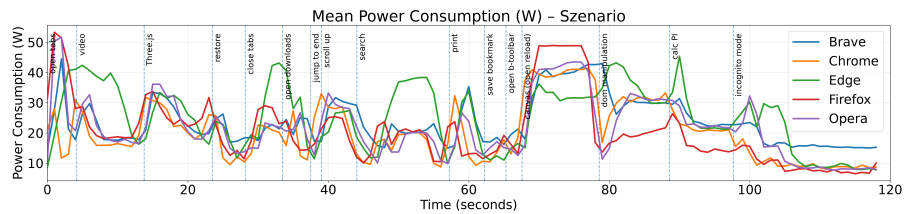


Fig. 5: Mean electrical power consumption of the five browsers during the standardized usage scenario.

Overall, each browser exhibits a characteristic power consumption profile, differing in both peak magnitude and temporal stability.

3.3 CPU Utilization

CPU utilization differed only moderately across the browsers during the standardized usage scenario. Microsoft Edge showed the highest mean CPU utilization, followed by Opera and Mozilla Firefox, while Google Chrome and Brave had the lowest values. Overall, the temporal profile of CPU load followed the sequence of user interactions, with clear peaks during video playback, tab management, search operations, and DOM-intensive actions.

Brave showed pronounced peaks during several interaction phases, whereas Opera and Edge maintained a comparatively higher baseline across the scenario. Firefox and Chrome generally remained below the other browsers, although all browsers exhibited short spikes during the most demanding tasks. Taken together, the CPU results indicate browser-specific workload handling, but the absolute differences remain relatively small.

Table 2: Mean CPU utilization of the browsers during the standardized usage scenario. Corrected values are calculated by subtracting the baseline CPU utilization of 4.1%.

Browser	Scenario [%]	Corrected [%]
Google Chrome	7.0 ± 0.2	2.9
Brave	7.1 ± 0.4	3.0
Mozilla Firefox	8.0 ± 0.2	3.9
Opera	9.1 ± 0.3	5.0
Microsoft Edge	10.6 ± 0.8	6.5

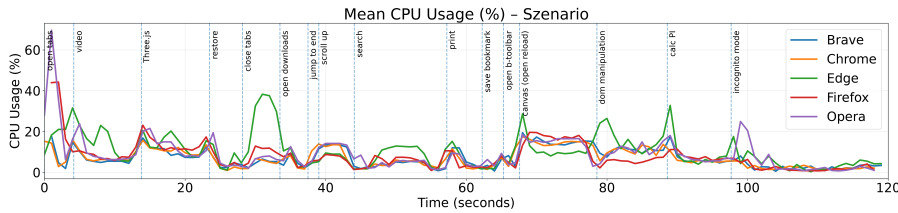


Fig. 6: CPU utilization of the five browsers over time during the standardized usage scenario. Vertical markers indicate individual user interactions.

3.4 Memory (RAM) and Disk Usage

Memory usage showed a clearer separation between browsers than CPU utilization. Mozilla Firefox consumed the most RAM during the scenario, while Google Chrome

and Brave used the least. Opera and Microsoft Edge remained in the middle range. The temporal development of RAM usage shows that memory demand increased strongly during the early loading phase, especially during the 3D animation, and then stabilized at a higher level for the remainder of the scenario.

Disk usage differed substantially across browsers and was the most pronounced resource-related contrast in this analysis. Opera generated by far the highest disk activity, while Chrome remained consistently low. Brave and Edge also showed elevated disk usage compared with Firefox, indicating that disk activity was not directly aligned with memory usage. The high standard deviation across individual test runs indicates considerable variability between runs and limits the conclusiveness of the results. Overall, RAM and disk measurements suggest that browsers differ not only in how much memory they reserve, but also in how aggressively they access storage during interaction-heavy workloads.

Table 3: Mean RAM usage of the browsers during the standardized usage scenario (mean $\pm$ SD, $n = 20$). Corrected values are calculated by subtracting the baseline RAM usage of 3.4 GB.

Browser	RAM [GB]	Corrected RAM [GB]
Google Chrome	3.74 ± 0.02	0.34
Brave	3.49 ± 0.04	0.09
Microsoft Edge	4.00 ± 0.05	0.60
Opera	4.15 ± 0.06	0.75
Mozilla Firefox	4.49 ± 0.01	1.09

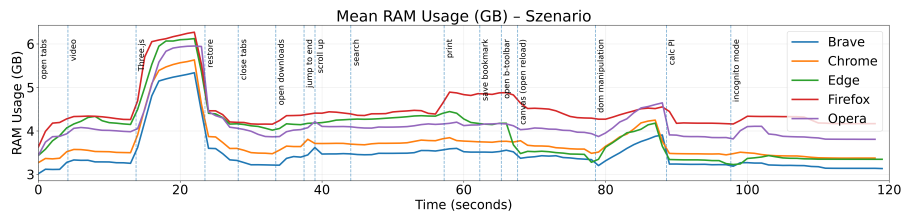


Fig. 7: RAM usage of the five browsers over time during the standardized usage scenario. Vertical markers indicate individual user interactions.

3.5 Network Traffic

Network traffic varied across browsers in both total data volume and DNS request frequency. Since the test website was hosted locally, the measured traffic primarily reflects

Table 4: Total disk activity of the browsers during the standardized usage scenario (mean $\pm$ SD, $n = 20$). Corrected values are calculated by subtracting the baseline disk activity of 514.2 Bytes/s.

Browser	Disk [Bytes/s]	Corrected Disk [Bytes/s]
Google Chrome	10167.09 $\pm$ 26018.41	9652.89
Mozilla Firefox	20295.23 $\pm$ 47568.27	19781.03
Microsoft Edge	25221.01 $\pm$ 34740.00	24706.81
Brave	50262.54 $\pm$ 164532.03	49748.34
Opera	305316.42 $\pm$ 391286.99	304802.22

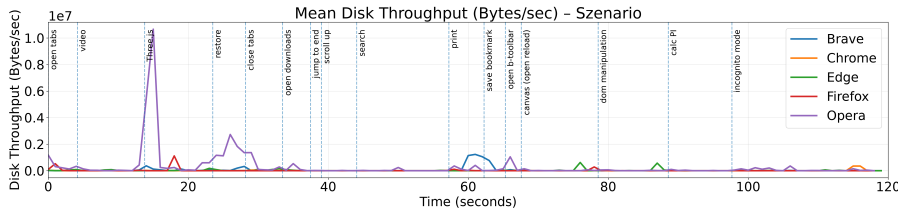


Fig. 8: Disk throughput of the five browsers over time during the standardized usage scenario. Vertical markers indicate individual user interactions.

communication with external endpoints rather than the test-page content itself. Firefox generated the highest average network traffic (68.12 Bytes/s), whereas Edge showed the lowest (45.51 Bytes/s). Edge transmitted 9.68 Bytes/s (21%) and received 35.83 Bytes/s (79%), while Chrome transmitted 7.46 Bytes/s (14%) and received 46.21 Bytes/s (86%).

DNS measurements showed a different ranking: Firefox generated the most requests (212.8 per run), followed by Edge (124.8), whereas Brave generated only 16.8 requests despite comparatively high network traffic. Brave also showed the highest mean data volume per request, while Firefox showed the lowest. These results indicate that network traffic volume and DNS request frequency capture different aspects of browser communication.

This divergence between data volume and DNS request frequency can be explained by the different characteristics of the two metrics. DNS request counts reflect the number of hostnames contacted by the browser and may include communication associated with background services, such as telemetry, update checks, synchronization, or other browser services. Network traffic, in contrast, measures the total amount of data exchanged, including both transmitted and received data, regardless of the number of endpoints involved. Firefox’s combination of relatively high DNS request activity and its network traffic therefore indicates that the browser contacted a comparatively large number of external endpoints, while the amount of data exchanged per connection remained relatively small.

Because individual requests could not be reliably assigned to specific categories, such as telemetry, advertising, synchronization, or content delivery, the underlying causes of these differences cannot be determined from the present measurements. Nevertheless,

network activity is relevant to software resource and energy efficiency and is considered in the Blue Angel criteria for software products [8]. According to [35], fewer requests with larger data volumes can be more energy-efficient than many smaller requests. From this perspective, Brave’s observed network behavior appears comparatively favorable, while Firefox generated substantially more DNS requests with less data per request. However, these observations should not be interpreted as direct evidence of differences in browser energy consumption.

Table 5: Mean network traffic for each browser during the standardized usage scenario. Corrected values are calculated by subtracting the baseline traffic of 0.193 Bytes/s.

Browser	Network [Bytes/s]	Corr. Network [Bytes/s]	send [Bytes/s]	received [Bytes/s]
Microsoft Edge	45.71 ± 4.07	45.51	9,68 (21%)	35,83 (79%)
Opera	52.29 ± 18.77	52.09	1,62 (2%)	50,47 (98%)
Google Chrome	53.86 ± 18.73	53.67	7,46 (14 %)	46,21 (86%)
Brave	61.06 ± 51.86	60.87	1,18 (2%)	59,69 (98%)
Mozilla Firefox	68.31 ± 9.08	68.12	2,85 (4%)	65,26 (96%)

Table 6: Mean number of DNS requests per run for each browser during the standardized usage scenario. Corrected values are calculated by subtracting the baseline of 0.85 requests per run.

Browser	DNS [No. of req.]	Corrected DNS [No. of req.]	mean Byte/request
Brave	17.6 ± 0.8	16.8	436,10
Google Chrome	27.7 ± 1.0	26.9	239,87
Opera	33.1 ± 1.2	32.3	193,84
Microsoft Edge	125.6 ± 3.7	124.8	43,78
Mozilla Firefox	213.6 ± 9.0	212.8	38,42

4 Discussion

The results demonstrate that browser efficiency cannot be assessed using a single resource metric. While CPU utilization differed only slightly between browsers, more pronounced differences were observed in electrical power consumption as well as memory, disk, and network activity. Within the standardized workload, Google Chrome exhibited the lowest mean power consumption, whereas Microsoft Edge showed the highest. However, the difference between Chrome and Firefox was comparatively small and should therefore be interpreted only within the investigated scenario. The measurements further show that browsers differ not only in the amount but also in the type of

resources they utilize. Firefox shows higher memory consumption and network activity, whereas Opera generated substantially higher disk activity than all other browsers. Although the underlying mechanisms were not investigated, this may indicate differences in caching, session management, or other storage-related operations that could be explored in future work.

Idle measurements revealed browser-specific background activity even without user interaction. In particular, Brave exhibited the highest idle network activity and power consumption, while Opera again showed elevated disk activity. Although the responsible processes cannot be identified from the present measurements, these observations indicate that background activity may contribute to the observed resource consumption. Finally, the results show that browsers sharing the Chromium rendering engine do not necessarily exhibit similar resource utilization. Despite their common architecture, Chrome, Edge, Opera, and Brave differed across several metrics, indicating that implementation-specific optimizations and integrated browser features can influence both resource utilization and electrical power consumption.

4.1 Contextualization of the Results

The results are consistent with previous studies reporting measurable differences in energy consumption and resource utilization between web browsers [36, 5, 32]. Compared with existing work, this study combines external electrical power measurements with synchronized monitoring of CPU, memory, disk, and network utilization under reproducible laboratory conditions. Furthermore, the inclusion of Pi-hole measurements provides additional insight into browser background communication, an aspect that is rarely considered in browser energy studies.

Although the observed differences in mean power consumption within the investigated scenario are moderate in absolute terms, their cumulative significance becomes apparent when considered at scale. The gap between the most efficient browser (Chrome, 20.4 W) and the least efficient (Edge, 25.2 W) amounts to approximately 4.8 W under the standardized workload. Assuming a typical daily active browsing duration of two hours, this difference corresponds to an additional energy expenditure of roughly 9.6 Wh per user per day, or approximately 3.5 kWh per year. Scaled to the estimated number of Microsoft Edge users worldwide, currently reported to exceed 300 million active monthly users [19], this translates to a potential cumulative difference of approximately 1.05 TWh per year compared with the most efficient browser. Using a global mean grid emission factor of approximately 436 g CO₂/kWh [13], this corresponds to a potential reduction of roughly 458,000 metric tons of CO₂-equivalent emissions annually. It should be emphasized that this figure represents a theoretical upper bound rather than a realistic savings estimate: it assumes that every Edge user would otherwise run Chrome under an identical workload, and the two-hour daily usage duration is an assumption rather than an empirically grounded figure for this user population. Real-world adoption patterns, hardware heterogeneity, and the fact that browser choice is rarely energy-driven mean that the actual achievable reduction would be substantially smaller. The estimate is presented solely to illustrate the order of magnitude of

potential energy savings achievable through browser-level efficiency differences, not as a projected outcome. From the perspective of Green Software Engineering, the results support the importance of considering application-level resource utilization when evaluating software energy efficiency [23, 14].

4.2 Critical Evaluation of the Results

The standardized measurement environment provides a high degree of reproducibility and enables a direct comparison between browsers. Nevertheless, several limitations should be considered when interpreting the results. First, the experiments were conducted in a controlled local environment. While this minimizes external influences and improves comparability, it cannot fully represent real-world browsing conditions involving dynamic web content, varying network conditions, advertisements, third-party services, and user-specific dynamic content. Consequently, the reported values should be interpreted as representative of the standardized measurement scenario rather than of general browser usage. Second, despite the use of a streamlined operating system (Tiny11), residual background processes cannot be eliminated. Although their influence is expected to be small, they may introduce minor variations into the measurements. Third, the presented results are based on external observations of system behavior. They reveal differences in resource consumption but do not identify the internal browser mechanisms responsible for these differences. Therefore, the measurements allow comparisons of browser behavior but do not explain the underlying implementation details. Fourth, the Measurement Client was equipped with 8 GB of main memory, of which approximately 3.4 GB was occupied by the baseline system. Under the more memory-intensive workloads, the system therefore operated closer to memory pressure, and part of the observed disk activity—particularly Opera’s elevated values—could be influenced by paging effects rather than browser-internal storage behavior alone. Fifth, although descriptive statistics including standard deviations were computed for all metrics, the present comparisons are based on mean values; a formal statistical significance analysis (e.g., paired t-tests or non-parametric equivalents such as the Wilcoxon signed-rank test across the 20 runs per browser) was not performed and is left for future work to substantiate the observed differences beyond measurement variance.

Finally, corrected values close to zero or slightly below zero should not be interpreted as physically negative resource consumption. Instead, they result from subtracting the corresponding baseline measurement and reflect normal measurement uncertainty together with unavoidable background activity.

5 Conclusion and Future Work

5.1 Conclusion

This study investigated the energy consumption and resource utilization of five widely used desktop web browsers using a reproducible measurement framework based on synchronized electrical power and system resource measurements.

Regarding **RQ1**, the results show measurable differences in electrical power consumption among browsers under the standardized usage scenario: Chrome was the most efficient and Edge the least efficient, although the gap to Firefox was small and scenario-specific. Regarding **RQ2**, no single system metric was sufficient to explain the observed power-consumption profiles; CPU utilization differed only marginally, while memory, disk, network, and idle behavior showed larger, browser-specific patterns—most notably Opera’s elevated disk activity and Firefox’s higher memory and network use. Regarding **RQ3**, the idle measurements demonstrated that browsers remain active even without user interaction, with Brave showing the highest idle network activity and power consumption, indicating that background processes contribute measurably to overall resource demand.

Overall, the results demonstrate the importance of combining electrical power measurements with multiple system metrics when evaluating browser energy efficiency. The proposed measurement framework provides a reproducible basis for comparing browser implementations and contributes to the broader objectives of Green Software Engineering.

5.2 Future Work

Future work should complement the presented black-box measurements with browser-internal profiling (white-box analysis) to identify the mechanisms responsible for the observed differences in resource utilization. In particular, Opera’s elevated disk activity and Brave’s increased idle network activity warrant further investigation.

In addition, extending the measurement setup with graphics processing unit (GPU) utilization, longer idle measurements, and realistic web workloads would provide a more comprehensive assessment of browser energy efficiency. Applying the framework to different operating systems, browser configurations, or mobile platforms would further improve the generalizability of the results.

Acknowledgments

The project presented was funded by the German Federal Environmental Foundation (DBU). **DBU Project 39703/01**

References

- [1] Actiona Project. Actiona, 2025.
- [2] Dlzar Al Kez, Aoife M. Foley, David Laverty, Dylan Furszyfer Del Rio, and Benjamin Sovacool. Exploring the sustainability challenges facing digitalization and internet data centers. *Journal of Cleaner Production*, 371:133633, 2022.
- [3] Brave Software, Inc. Brave browser. <https://brave.com/>, January 2026. Version 1.85.120.
- [4] CHIP Digital GmbH. Chip download charts – browser, 2026. Accessed on 31 January 2026.
- [5] Anthony DiBenedetto and Fadi Wedyan. An empirical evaluation of energy consumption across web browsers. In *2025 10th International Conference on Fog and Mobile Edge Computing (FMEC)*, pages 39–48. IEEE, 2025.
- [6] Markus Dick, Eva Kern, Jakob Drangmeister, Stefan Naumann, and Timo Johann. Measurement and rating of software-induced energy consumption of desktop pcs and servers. In Werner Pillmann, Sven Schade, and Paul Smits, editors, *Innovations in Sharing Environmental Observations and Information. Proceedings of the 25th International Conference EnviroInfo, October 5–7, 2011, Ispra, Italy*, pages 290–299, Aachen, 2011. Shaker.
- [7] Markus Dick, Eva Kern, Timo Johann, Stefan Naumann, and Christian Glden. Green web engineering – measurements and findings. In Hans-Knud Arndt, Gerlinde Knetsch, and Werner Pillmann, editors, *EnviroInfo 2012: Man – Environment – Bauhaus. Light up the Ideas of Environmental Informatics*, pages 599–606, Aachen, 2012. Shaker.
- [8] German Environment Agency. Blue angel ecolabel – resource and energy-efficient software products (de-uz 215). www.blauer-engel.de/uz215, 2026. Edition January 2020, Version 6.
- [9] Google LLC. Google chrome. <https://www.google.com/chrome/>, December 2025. Version 143.0.7499.169/170.
- [10] Green Software Engineering Group. `get_energy_utilizationgraph_mit_sync.py`, 2026.
- [11] Jens Grger, Andreas Khler, Stefan Naumann, Andreas Filler, Achim Guldner, Eva Kern, Lorenz Hilty, and Yuliy Maksimov. Entwicklung und anwendung von bewertungsgrundlagen fr ressourceneffiziente software unter bercksichtigung bestehender methodik-abschlussbericht. *UBA TEXTE*, 105:39–40, 2018.
- [12] GUDE Systems. Expert power control 1202: 4-fold switched and metered pdu, 2026.
- [13] International Energy Agency. Energy efficiency 2023, 2023.
- [14] Dennis Junger, Max Westing, Christopher P. Freitag, Achim Guldner, Konstantin Mittelbach, Kira Obergcker, Sebastian Weber, Stefan Naumann, and Volker Wohlgemuth. Potentials of green coding – findings and recommendations for industry, education and science – extended paper. *arXiv preprint arXiv:2402.18227*, 2024.
- [15] Dennis Junger and Volker Wohlgemuth. Current development trends in environmental informatics: An educational course, 2023.

- [16] Dennis Junger and Volker Wohlgemuth. Design and implementation of a lecture for teaching current green coding approaches and practices at the htw berlin, 2023.
- [17] Dennis Junger, Volker Wohlgemuth, and Eike Kammer. Conception and test of a measuring station for the analysis of the resource and energy consumption of material flow oriented environmental management information systems (emis). In *EnviroInfo 2022*, page 211. Gesellschaft für Informatik eV, 2022.
- [18] Fabian Kleinert and Hans-Knud Arndt. Energy efficiency in web development: Investigation of the power consumption of a web application with different load distribution. In *Advances and New Trends in Environmental Informatics. EnviroInfo 2022*, Progress in IS. Springer, Cham, 2023.
- [19] Maitrayee Dey. Microsoft edge statistics by market share and trends, 2025. Updated 2025-08-30.
- [20] Jens Malmodin, Nina Lövehagen, Pernilla Bergmark, and Dag Lundén. Ict sector electricity consumption and greenhouse gas emissions – 2020 outcome. *Telecommunications Policy*, 48(3):102701, 2024.
- [21] Microsoft. Microsoft edge. <https://www.microsoft.com/edge>, December 2025. Version 143.0.3650.96.
- [22] Mozilla Foundation. Mozilla firefox. <https://www.mozilla.org/firefox/>, December 2025. Version 146.0.1.
- [23] Stefan Naumann, Markus Dick, Eva Kern, and Timo Johann. The greensoft model: A reference model for green and sustainable software and its engineering. *Sustainable Computing: Informatics and Systems*, 1(4):294–304, 2011.
- [24] NTDEV. tiny11builder. <https://github.com/ntdevlabs/tiny11builder>, 2023. GitHub repository, accessed 2026-07-16.
- [25] HTW Berlin University of Applied Sciences, Dennis Junger, Marie Charlotte Epp, Kevin Jacob Bärwald, Nicolaus Lea, Ramona Pangerl, and Stefan Schlegel. Green browsing – comparing energy consumption of modern web browsers: Brave, July 2026.
- [26] HTW Berlin University of Applied Sciences, Dennis Junger, Marie Charlotte Epp, Kevin Jacob Bärwald, Nicolaus Lea, Ramona Pangerl, and Stefan Schlegel. Green browsing – comparing energy consumption of modern web browsers: Opera, July 2026.
- [27] HTW Berlin University of Applied Sciences, Dennis Junger, Marie Charlotte Epp, Kevin Jacob Bärwald, Lea Nicolaus, Ramona Pangerl, and Stefan Schlegel. Green browsing – comparing energy consumption of modern web browsers: Edge, July 2026.
- [28] HTW Berlin University of Applied Sciences, Dennis Junger, Marie Charlotte Epp, Kevin Jacob Bärwald, Lea Catharina Nicolaus, Ramona Pangerl, and Stefan Schlegel. Green browsing – comparing energy consumption of modern web browsers: Baseline, June 2026.
- [29] HTW Berlin University of Applied Sciences, Dennis Junger, Marie Charlotte Epp, Kevin Jacob Bärwald, Lea Catharina Nicolaus, Ramona Pangerl, and Stefan Schlegel. Green browsing – comparing energy consumption of modern web browsers: Chrome, June 2026.
- [30] HTW Berlin University of Applied Sciences, Dennis Junger, Marie Charlotte Epp, Kevin Jacob Bärwald, Lea Catharina Nicolaus, Stefan Schlegel, and Ra-

- mona Pangerl. Green browsing – comparing energy consumption of modern web browsers: Firefox, July 2026.
- [31] Opera Norway AS. Opera. <https://www.opera.com/>, January 2026. Version 126.0.5750.43.
 - [32] Nadja Peters, Sangyoung Park, Daniel Clifford, Sami Kyostila, R McIlroy, Benedikt Meurer, Hannes Payer, and Samarjit Chakraborty. Phase-aware web browser power management on hmp platforms. In *Proceedings of the 2018 International Conference on Supercomputing*, pages 274–283, 2018.
 - [33] United Nations Publications. *Digital Economy Report 2024*. UN, 9 2024.
 - [34] Selenium. Webdriver, 2026.
 - [35] Valters Skrastins, Vladislavs Medvedevs, Dmitrijs Orlovs, Juris Ormanis, and Janis Judvaitis. Experimental evaluation of nb-iot power consumption and energy source feasibility for long-term iot deployments. *IoT*, 7(1), 2026.
 - [36] Asoke K. Talukder, Lawrence Zimmerman, and Prahalad H. A. Cloud economics: Principles, costs, and benefits, 2010.